

INT301 Lecture Summary

考试终极知识点清单

Contents

1 Lecture 1: 生物计算基石: M-P 神经元模型的离散计算与 Hebb 规则的相关性学习	9
1.1 模块一: M-P 神经元模型 (The “Hardware”)	9
1.1.1 核心定义与属性	9
1.1.2 激活函数 (Threshold Activation Function)	9
1.1.3 计算逻辑 (运作流程)	10
1.2 模块二: 人工神经网络的学习定义 (The Concept)	10
1.2.1 学习的本质	10
1.3 模块三: Hebb 学习规则 (The “Algorithm”)	10
1.3.1 核心假设 (The Postulate)	10
1.3.2 数学公式 (考试必考)	11
1.3.3 致命缺陷 (理论考点)	11
1.4 模块四: Hebb 规则计算套路 (Calculation Recipe)	11
1.4.1 步骤 1: 前向传播 (计算输出)	11
1.4.2 步骤 2: 计算权重增量 (ΔW)	11
1.4.3 步骤 3: 更新权重 (W_{new})	12
1.4.4 步骤 4: 迭代与累积 (重要考点)	12
1.5 考试避坑指南 (易错点)	12
2 Lecture 2: 监督学习与感知机 (Supervised Learning & Perceptron)	13
2.1 第一板块: 核心概念与定义 (Concepts & Definitions)	13
2.1.1 监督学习 (Supervised Learning)	13
2.1.2 学习的基本假设 (Fundamental Assumption) [高频考点]	13
2.1.3 感知机 (Perceptron)	13
2.2 第二板块: 感知机架构与数学模型 (Architecture & Math)	13

2.2.1	网络结构	14
2.2.2	核心计算公式	14
2.3	第三板块: 核心算法 — Delta 规则 (The Algorithm & Calculation) . . .	14
2.3.1	误差计算 (Error Calculation)	14
2.3.2	权重更新公式 (Delta Rule) [必背]	15
2.3.3	更新逻辑判断 (Logic Check)	15
2.3.4	训练终止条件 (Convergence Condition)	15
2.4	第四板块: 几何意义与收敛性 (Geometry & Theory)	15
2.4.1	决策边界 (Decision Boundary)	15
2.4.2	感知机收敛定理 (Perceptron Convergence Theorem)	16
2.4.3	性能度量 (RMS Error)	16
2.5	第五板块: 线性可分性与局限性 (Linear Separability & Limitations) . .	16
2.5.1	线性可分 (Linearly Separable)	16
2.5.2	线性不可分 (Linearly Inseparable) — XOR 问题 [核弹级考点] . .	16
2.5.3	解决办法	17
2.6	考前 Check List	17
3	Lecture 3: 梯度下降与 Sigmoid 感知机	18
3.1	第一板块: 核心概念与理论基础 (Theory & Concepts)	18
3.1.1	感知机的局限性 (The Limitation)	18
3.1.2	梯度下降的目标 (Objective of Gradient Descent)	18
3.2	第二板块: 核心公式与推导 (Formulas & Derivations)	18
3.2.1	线性单元 (Linear Unit) 的梯度下降	19
3.2.2	Sigmoid 单元 (Sigmoid Unit) 的梯度下降	19
3.3	第三板块: 算法流程与计算套路 (Algorithm & Calculation)	19
3.3.1	批量梯度下降 (Batch Gradient Descent) / 标准梯度下降	19
3.3.2	增量/随机梯度下降 (Incremental / Stochastic Gradient Descent — SGD)	20
3.3.3	手算题常见坑点 (Common Pitfalls)	20
3.4	第四板块: 理论对比总结 (Comparison)	20

3.5	考前 Checklist	21
4	Lecture 4: 多层感知机 (MLP) 与反向传播 (BP)	22
4.1	第一模块: MLP 基础架构与核心组件	22
4.1.1	MLP 的核心定义	22
4.1.2	激活函数 (Activation Functions)	22
4.1.3	表示能力 (Representation Power)	22
4.2	第二模块: 反向传播算法 (Backpropagation) — 考试绝对核心	23
4.2.1	算法核心逻辑	23
4.2.2	完整的计算步骤 (必须要背下来的算法流程)	23
4.2.3	核心计算公式 (死记硬背区)	23
4.3	第三模块: 训练策略与调优 (Hyperparameters & Tuning)	24
4.3.1	训练模式	24
4.3.2	关键参数设置	24
4.3.3	动量项 (Momentum) — 必考优化技巧	25
4.4	第四模块: 过拟合与泛化 (Overfitting & Generalization)	25
4.4.1	现象与原因	25
4.4.2	解决方案 (三大法宝)	25
4.5	第五模块: MLP 的能力总结	26
4.6	考前最后叮嘱	26
5	Lecture 5: 卷积神经网络 (CNN) 最终冲刺	27
5.1	第一板块: 核心架构理念 (Core Philosophy)	27
5.1.1	端到端学习 (End-to-end Learning)	27
5.1.2	特征层级结构 (Feature Hierarchy)	27
5.1.3	CNN 的三大核心思想 (Key Ideas) [高频考点]	27
5.2	第二板块: 核心数学公式与计算 (Formulas & Calculations)	27
5.2.1	卷积输出尺寸计算 [必背]	27
5.2.2	卷积层计算逻辑	27
5.2.3	激活函数 (Activation Functions)	28

5.2.4	优化算法更新 (SGD)	28
5.3	第三板块: 关键组件对比与选型逻辑 (Analysis & Comparison)	28
5.3.1	ReLU vs. Sigmoid (为什么选 ReLU?) [高频考点]	28
5.3.2	池化操作 (Pooling)	28
5.3.3	归一化 (Normalization)	28
5.4	第四板块: 训练与进阶策略 (Training & Advanced)	29
5.4.1	反向传播 (Back-propagation) [核心考点]	29
5.4.2	迁移学习 (Transfer Learning) [高频考点]	29
5.5	考前寄语	29
6	Lecture 6: 径向基函数网络 (RBFN) 与聚类分析	30
6.1	第一板块: RBFN 核心理论与设计直觉 (Concept & Rationale)	30
6.1.1	基本定义	30
6.1.2	核心直觉 (The Rationale)	30
6.1.3	正则化网络 vs. RBF 网络 (必须分清)	30
6.2	第二板块: 网络架构与数学模型 (Architecture & Math)	30
6.2.1	三层网络结构	31
6.2.2	网络输出公式 (单样本)	31
6.3	第三板块: RBF 学习算法与权重计算 (The “Must-Memorize” Formulas)	31
6.3.1	训练目标	31
6.3.2	设计矩阵 (Design Matrix) Φ	31
6.3.3	权重求解公式 (The Pseudo-Inverse Method) [必背]	32
6.3.4	RBF 完整训练流程	32
6.4	第四板块: 中心点选择与 K-means 聚类 (Center Selection & Algorithms)	32
6.4.1	选择方法	32
6.4.2	K-means 算法流程 (闭眼都要能背出步骤)	32
6.4.3	距离度量	32
6.4.4	K-means 的缺陷 (Issues) [简答题高频考点]	33
6.5	第五板块: 概念大对决 RBF vs MLP (Comparisons)	33
6.6	考前最后 Check	33

7	Lecture 7: 时间序列与 Elman 网络 (Time Series & Elman Network)	34
7.1	第一板块: 时间序列基础理论 (Time Series Fundamentals)	34
7.1.1	核心定义 (Key Definition)	34
7.1.2	统计属性与平稳性 (Statistical Properties & WSS)	34
7.2	第二板块: 预测问题的数学建模 (Prediction Problem Formulation)	34
7.2.1	预测公式	34
7.2.2	建模三部曲 (Standard Procedure)	35
7.2.3	预测的难点 (Difficulties)	35
7.3	第三板块: 静态网络与滑动窗口 (Static Networks: TDNN)	35
7.3.1	滑动窗口机制 (Sliding Window)	35
7.3.2	时间延迟网络 (Time Delayed Networks, TDNN)	35
7.4	第四板块: 动态神经网络基础 (Dynamical/Recurrent Networks)	36
7.4.1	静态 vs 动态 (Static vs Dynamic)	36
7.4.2	核心特征	36
7.5	第五板块: Elman 网络 (The Elman Network)	36
7.5.1	定义与归类	36
7.5.2	四层架构 (Architecture)	36
7.5.3	数学方程 (Equations) [必须背诵]	37
7.5.4	训练算法	37
7.6	考前自测问题 (Self-Check)	37
8	Lecture 8: 循环神经网络与 LSTM (RNN & LSTM)	38
8.1	第一板块: RNN 的核心逻辑与应用场景 (The “Why” & “What”)	38
8.1.1	核心动机 (Motivation)	38
8.1.2	五大架构类型与考题映射 (Architectures & Examples)	38
8.1.3	拓扑结构概念	38
8.2	第二板块: RNN 数学原理与训练 (The Math & Algorithm)	38
8.2.1	RNN 前向传播公式 (Forward Pass)	39
8.2.2	损失函数 (Loss Function)	39
8.2.3	训练算法: BPTT (Back Propagation Through Time)	39

8.2.4	核心问题: 梯度消失 (Vanishing Gradients)	39
8.3	第三板块: LSTM 深度解析 (The Solution)	40
8.3.1	设计初衷	40
8.3.2	LSTM 细胞的三大信号流 (Gates Logic)	40
8.3.3	LSTM 完整计算流程 [公式必背]	40
8.4	第四板块: LSTM 优缺点评估 (Critical Evaluation)	41
8.4.1	优势 (Advantages)	41
8.4.2	局限性与风险 (Limitations)	41
8.5	考前 1 分钟自测 (Checklist)	42
9	Lecture 9: 主成分分析 (Introduction to PCA)	43
9.1	第一板块: 数学基石 (Linear Algebra Foundations)	43
9.1.1	特征值与特征向量 (Eigenvalues & Eigenvectors)	43
9.1.2	实对称矩阵 (Real Symmetric Matrices)	43
9.2	第二板块: 矩阵分解核心 (Matrix Decomposition)	43
9.2.1	特征分解 (Eigen Decomposition / Spectral Decomposition)	43
9.2.2	奇异值分解 (SVD)	44
9.3	第三板块: PCA 理论推导逻辑 (Theory & Derivation)	44
9.3.1	核心目标	44
9.3.2	推导链条 (逻辑必须清晰)	44
9.4	第四板块: PCA 计算流程与算法 (Algorithm Logic)	45
9.4.1	步骤 1: 数据预处理	45
9.4.2	步骤 2: 构建协方差矩阵	45
9.4.3	步骤 3: 特征值分解	45
9.4.4	步骤 4: 选择主成分 (降维)	45
9.4.5	步骤 5: 投影 (计算新坐标)	46
9.5	第五板块: 批判性分析 (Critical Analysis)	46
9.5.1	局限性 (Limitations)	46
9.5.2	特征脸 (Eigenfaces) 案例	46
9.6	考前叮嘱 (Final Tips)	46

10 Lecture 10: 无监督学习 (Unsupervised Learning)	47
10.1 第一板块: 赫布学习与 Oja 规则 (基础与修正)	47
10.1.1 赫布学习 (Hebbian Learning)	47
10.1.2 Oja 规则 (Oja's Rule)	47
10.1.3 紧缩法 (Deflation Method)	48
10.2 第二板块: 神经网络中的 PCA 与自动编码器	48
10.2.1 自动编码器 (Auto-encoders)	48
10.2.2 变体模型	48
10.3 第三板块: 聚类与 K-means	49
10.3.1 K-means 算法流程 (背诵级)	49
10.3.2 核心考点	49
10.4 第四板块: 竞争学习 (Competitive Learning)	49
10.4.1 赢家通吃 (Winner-Takes-All, WTA)	49
10.4.2 “死单元”问题 (Dead Units) 与解决方案	50
10.4.3 Maxnet (WTA 的网络实现)	50
10.4.4 墨西哥帽网络 (Mexican Hat Network)	50
10.5 第五板块: 应用 — 矢量量化 (Vector Quantization)	50
10.5.1 核心概念	50
10.6 考前叮嘱 (Exam Tips)	51
11 Lecture 11: 自组织映射与学习矢量量化 (SOM & LVQ)	52
11.1 第一板块: 核心概念与生物学原理 (The Concept)	52
11.1.1 核心目标 (Goal)	52
11.1.2 生物学机制 (Mexican Hat Function)	52
11.2 第二板块: SOM 算法流程与计算 (The Algorithm)	52
11.2.1 网络架构 (Architecture)	52
11.2.2 训练三大步 (The 3 Processes)	52
11.3 第三板块: SOM 的特性与结果 (Properties)	53
11.3.1 近似输入空间 (Approximation)	53
11.3.2 密度匹配 (Density Matching)	54

11.3.3 特征选择 (Feature Selection)	54
11.4 第四板块: 矢量量化与 LVQ (VQ & LVQ)	54
11.4.1 矢量量化 (Vector Quantization, VQ)	54
11.4.2 学习矢量量化 (LVQ) — 有监督扩展	54
11.4.3 LVQ2 (进阶)	54
11.4.4 LVQ 优缺点	55
11.5 考前特别提醒 (Final Check)	55
12 Lecture 12: 联想记忆与离散 Hopfield 网络	56
12.1 第一板块: 联想记忆基础 (Associative Memory Basics)	56
12.1.1 核心定义与对比	56
12.1.2 两大分类 (必须要分清)	56
12.2 第二板块: 简单联想记忆与矩阵数学 (Simple AM & Math)	56
12.2.1 赫布学习规则 (Hebbian Learning Rule)	56
12.2.2 权重矩阵构建 (外积法 — Outer Product)	57
12.2.3 回忆过程与干扰 (Recall & Cross-talk)	57
12.3 第三板块: 离散 Hopfield 网络 (Discrete Hopfield Network)	57
12.3.1 网络架构特征 (背诵)	57
12.3.2 权重矩阵计算公式 (必考计算)	58
12.3.3 状态更新规则 (Update Rule / Dynamics)	58
12.4 第四板块: 系统性质与局限性 (Properties & Limits)	58
12.4.1 稳定性与收敛 (Convergence)	59
12.4.2 伪状态 (Spurious States)	59
12.4.3 存储容量 (Capacity)	59
12.5 第五板块: 考试解题 SOP (Standard Operating Procedure)	59
12.5.1 Step 1: 检查模式类型	59
12.5.2 Step 2: 构建 W 矩阵	59
12.5.3 Step 3: 进行迭代/回忆	60
12.5.4 Step 4: 验证稳定性	60

1 Lecture 1: 生物计算基石: M-P 神经元模型的离散计算与 Hebb 规则的相关性学习

1.1 模块一: M-P 神经元模型 (The “Hardware”)

这是所有计算的基础, 考试中常考定义判断和基础计算。

1.1.1 核心定义与属性

模型本质: 它是对生物神经元的抽象, 被建模为**二进制离散时间元件** (Binary discrete-time element)。

信号性质:

- 输入 (a_i): 只能是 0 或 1。
- 输出 (x_j): 只能是 0 或 1。
- 权重 (w_{ij}): 在原始 M-P 模型中, +1 代表兴奋性 (Excitatory), -1 代表抑制性 (Inhibitory)。

状态依赖性:

- 神经元当前的状态 (S^t) 只取决于当前的输入, 与之前的状态无关 (无记忆性, 除非通过权重变化)。
- 输出是状态的函数: $x(t) = g(S^t)$ 。

1.1.2 激活函数 (Threshold Activation Function)

数学形式: 使用 Heaviside (单位阶跃) 函数。

公式 (必须背诵):

$$g(S) = H(S - \theta) = \begin{cases} 1, & \text{if } S \geq \theta \\ 0, & \text{if } S < \theta \end{cases}$$

物理含义:

- S : 总加权输入。
- θ : 激发阈值 (Excitation Threshold)。
- 只有当总输入超过门槛, 神经元才“点火” (Fire, 输出 1)。

1.1.3 计算逻辑 (运作流程)

1. **求和**: 计算所有输入与对应权重的乘积之和。

$$S^t = \sum_i w_i a_i^t$$

2. **比较**: 将 S^t 与阈值 θ 比较。

3. **输出**: 决定下一时刻 $t + 1$ 的输出 x^{t+1} 。

1.2 模块二：人工神经网络的学习定义 (The Concept)

这部分主要涉及理论填空或选择题。

1.2.1 学习的本质

定义: 学习意味着**针对经验做出改变** (Change in response to experience)。

实现方式:

- 不改变连接模式 (太昂贵/复杂) 。
- **改变连接的权重 (w_{ij})**: 这是 ANN 中唯一的“自由参数” (Free parameters)。

目标: 调整权重, 使网络能更好地适应整体功能 (如存储知识、解决问题) 。

1.3 模块三: Hebb 学习规则 (The “Algorithm”)

这是本章节的计算题绝对核心, 必须能够默写公式并进行多轮迭代计算。

1.3.1 核心假设 (The Postulate)

名言: “Cells that fire together, wire together.” (一起激发的细胞, 连在一起)。

原理: 如果神经元 A 反复参与激发神经元 B, 那么它们之间的连接效率会增加。

性质:

- **相关性 (Correlation)**: 只有输入和输出同时为高 (1) 时, 连接才增强。
- **局部性**: 权重的更新只依赖于该连接两端的信号, 不需要全局信息。
- **无监督 (Unsupervised)**: 不需要“导师”告诉网络正确答案, 它根据输入模式自组织。

1.3.2 数学公式 (考试必考)

权重更新公式:

$$w_{ji}^{k+1} = w_{ji}^k + \Delta w_{ji}^k$$

增量计算公式:

$$\Delta w_{ji}^k = C \cdot a_i^k \cdot x_j^k$$

变量解释:

- k : 当前时间步。
- C : 学习率 (Learning rate), 正系数, 通常设为 1。
- a_i : 突触前输入值 (Pre-synaptic input)。
- x_j : 突触后输出值 (Post-synaptic output)。注意: 这是计算出来的实际输出, 不是期望输出!

1.3.3 致命缺陷 (理论考点)

问题: 权重只能增加 (Only increase)。

后果: 随着时间推移, 权重会无限增长, 导致网络进入饱和 (Saturation) 状态, 所有神经元都对所有刺激有反应, 失去区分能力。

解决思路 (提及): 引入归一化 (Normalization) 或遗忘机制。

1.4 模块四: Hebb 规则计算套路 (Calculation Recipe)

针对计算大题的“万能解题步骤”, 请严格按此步骤书写步骤分。

场景: 给定初始权重 W^0 、阈值 θ 、学习率 C 和一系列输入模式 (Pattern A, Pattern B...)。

1.4.1 步骤 1: 前向传播 (计算输出)

1. 计算当前模式下的总输入: $S = \sum a_i w_i$
2. 判断是否激活: 若 $S \geq \theta$, 则 $X = 1$; 否则 $X = 0$ 。

注意: 如果 $X = 0$, 根据 Hebb 公式, Δw 将全为 0, 权重不更新。

1.4.2 步骤 2: 计算权重增量 (ΔW)

只有当 $X = 1$ 时才需要计算。

对于每一个输入节点 i :

- 若输入 $a_i = 1$ 且 $X = 1 \rightarrow \Delta w_i = C \cdot 1 \cdot 1 = C$ 。
- 若输入 $a_i = 0 \rightarrow \Delta w_i = 0$ 。

1.4.3 步骤 3: 更新权重 (W_{new})

$$W_{new} = W_{old} + \Delta W$$

解释: 将刚才算出的增量加到旧权重上。

1.4.4 步骤 4: 迭代与累积 (重要考点)

如果题目给出第二个不同的输入模式:

- 必须使用步骤 3 算出的最新权重 W_{new} 作为起点。
- 重复步骤 1-3。

结果分析:

- 在多个模式中共同出现的特征（输入节点），其权重会叠加增长，变得最大。
- 只在某个模式出现的特征，权重增长较慢。
- 从未出现的特征，权重保持初始值。

1.5 考试避坑指南 (易错点)

- **输入值陷阱:** M-P 模型的输入通常是 0 或 1。如果题目给出 -1，请仔细确认是否是变体模型（但根据你的课件，坚持 0/1）。
- **权重更新时机:** 权重是在算出输出 X 之后才更新的，不是计算中途更新。
- **无输入不更新:** 即使 $X = 1$ ，如果某个特定节点 $a_i = 0$ （没信号），该节点的权重 w_i 不会增加（因为 $0 \times 1 = 0$ ）。
- **无激发不更新:** 如果总输入 $S < \theta$ ，导致 $X = 0$ ，那么所有权重都不变。

2 Lecture 2: 监督学习与感知机 (Supervised Learning & Perceptron)

2.1 第一板块：核心概念与定义 (Concepts & Definitions)

这部分主要应对选择题、填空题或简答题，考察对基本术语的理解。

2.1.1 监督学习 (Supervised Learning)

定义：从**带标签 (Labeled)** 的训练数据中学习模型的过程。数据通常表示为 (x, y) 对，其中 x 是输入特征， y 是预定义的类别 (Target)。

比喻：就像有一个“老师 (Teacher)”在旁边告诉你标准答案。

核心流程 (Two Steps):

1. **训练 (Training):** 使用训练数据 + 学习算法 $\rightarrow$ 生成模型 (Model)。
2. **测试 (Testing):** 使用未见过 (Unseen) 的测试数据 + 模型 $\rightarrow$ 评估准确率 (Accuracy)。

学习的定义 (Learning Definition):

如果系统在任务 T 上的性能 P 随着经验 E (数据) 的增加而提高，则称系统发生了学习。

2.1.2 学习的基本假设 (Fundamental Assumption) [高频考点]

内容：训练样本的分布必须与测试样本 (包括未来数据) 的分布**完全相同 (Identical)**。

考点：如果违背这个假设 (即训练数据无法代表测试数据)，模型的分类准确率会非常差。

2.1.3 感知机 (Perceptron)

历史：1958 年由 Rosenblatt 提出。

本质：一种用于二分类的线性分类模型，基于**纠错规则 (Error-correcting rule)** 进行学习。

2.2 第二板块：感知机架构与数学模型 (Architecture & Math)

这部分是计算题的基础，必须熟练掌握公式。

2.2.1 网络结构

- 输入层 (Input Units): $x_1, x_2, \dots, x_n$ 。
- 偏置单元 (Bias Unit): $x_0 = 1$ (恒定输入, **必须记住**)。
- 权重 (Weights): $w_0, w_1, \dots, w_n$ 。 w_0 是偏置的权重。
- 输出层 (Output Unit): 单个神经元。

2.2.2 核心计算公式

加权求和 (Total Input / Weighted Sum):

$$S_j = \sum_{i=0}^n w_{ji} a_i = \mathbf{w} \cdot \mathbf{x}$$

(注意: 求和是从 $i = 0$ 开始的, 包含了偏置项)

激活函数 (Activation Function):

感知机使用的是阈值函数 (Threshold Function) 或阶跃函数。

判定规则:

$$X_j = \begin{cases} 1, & S_j \geq \theta \\ 0, & S_j < \theta \end{cases}$$

注: 有时阈值 θ 会被移项整合进偏置 w_0 中, 此时判断 $S_j \geq 0$ 即可。

2.3 第三板块: 核心算法 — Delta 规则 (The Algorithm & Calculation)

这是计算大题的绝对核心。 考试中通常会给你初始权重和样本, 让你手算几轮迭代。

2.3.1 误差计算 (Error Calculation)

$$e = t - o$$

- t : 目标值 (Target/Teacher), 正确答案。
- o : 实际输出值 (Output), 模型当前的预测。

可能的结果:

- 0: 预测正确。
- +1: 漏报 (False Negative), 预测是0, 实际是1。→ 需要增加权重。
- -1: 误报 (False Positive), 预测是1, 实际是0。→ 需要减少权重。

2.3.2 权重更新公式 (Delta Rule) [必背]

$$\Delta w_i = \eta \cdot (t - o) \cdot x_i$$

$$w_i^{new} = w_i^{old} + \Delta w_i$$

- η (Eta): 学习率 (Learning Rate), 通常 < 1 , 防止调整幅度过大。
- x_i : 对应权重的输入值。

2.3.3 更新逻辑判断 (Logic Check)

在计算时, 问自己三个问题, 决定是否更新:

1. 预测对了吗? 对了 ($e = 0$) $\rightarrow$ 不更新 ($\Delta w = 0$)。
2. 输入是 0 吗? 输入为 0 ($x_i = 0$) $\rightarrow$ 不更新 ($\Delta w = 0$)。(该特征未参与决策, 不背锅)。
3. 预测错了吗? 错了且输入为 1 $\rightarrow$ 更新。

2.3.4 训练终止条件 (Convergence Condition)

当完整遍历一遍所有训练样本 (Epoch), 且没有任何权重发生变化 (即误差全为 0) 时, 训练结束。

2.4 第四板块: 几何意义与收敛性 (Geometry & Theory)

这部分涉及图解题和理论判断题。

2.4.1 决策边界 (Decision Boundary)

感知机的权重向量 $\mathbf{w}$ 定义了一个超平面 (Hyperplane)。

方程:

$$\mathbf{w} \cdot \mathbf{x} = 0 \quad (\text{即 } w_0 + w_1x_1 + w_2x_2 = 0)$$

2D 几何意义:

- 这是一条直线。
- 权重向量 $\mathbf{w}$ 的方向与决策边界正交 (Orthogonal/Perpendicular)。
- 偏置 w_0 决定直线距离原点的位置 (Position) (截距)。
- 权重 w_1, w_2 决定直线的角度/斜率 (Orientation)。

2.4.2 感知机收敛定理 (Perceptron Convergence Theorem)

定理: 如果数据集是**线性可分 (Linearly Separable)** 的, 且学习率足够小, 感知机一定能在**有限步骤 (Finite steps)** 内找到解。

逆否命题: 如果数据不可分, 感知机将永远无法收敛 (会一直震荡)。

2.4.3 性能度量 (RMS Error)

训练过程中最小化均方根误差 (RMS):

$$RMS = \sqrt{\frac{\sum (t - o)^2}{N}}$$

- RMS 是权重的函数。
- 我们调整权重是为了找到 RMS 的最小值。

2.5 第五板块: 线性可分性与局限性 (Linear Separability & Limitations)

这部分通常是压轴的证明题或概念辨析题。

2.5.1 线性可分 (Linearly Separable)

能用一条直线 (或超平面) 将两类数据完全分开。

案例: Logical AND 和 Logical OR 都是线性可分的, 感知机可以解决。

2.5.2 线性不可分 (Linearly Inseparable) — XOR 问题 [核弹级考点]

XOR (异或): 当输入相同时输出 0, 不同时输出 1。

- (0,0) -> 0
- (1,1) -> 0
- (0,1) -> 1
- (1,0) -> 1

结论: 单层感知机无法解决 XOR 问题。

证明逻辑 (Proof Logic):

1. 列出 4 个不等式。
2. 尝试组合不等式 (例如 (1)+(4) 和 (2)+(3))。
3. 推导出矛盾结果 (例如 $b < 0$ 且 $b \geq 0$)。
4. 证明不存在这样的直线。

2.5.3 解决办法

单层不够，多层来凑。需要引入**多层感知机 (MLP)** 或更复杂的架构。

2.6 考前 Check List

在进考场前，请确保你能完成以下操作：

- 默写公式： S 的求和公式， Δw 的更新公式。
- 手动模拟：随便给自己出 3 个数据点，设定初始权重为 0，用 Delta Rule 手算两轮迭代。
- 画图：在纸上画出 AND, OR, XOR 的坐标点分布，并画线尝试分割它们。
- 背诵：“Fundamental Assumption of Learning” 和 “Perceptron Convergence Theorem” 的定义。
- 证明：能够复述为什么 XOR 不能被线性分割（不等式矛盾法）。

3 Lecture 3: 梯度下降与 Sigmoid 感知机

3.1 第一板块：核心概念与理论基础 (Theory & Concepts)

这部分主要应对选择题、判断题和简答题。

3.1.1 感知机的局限性 (The Limitation)

线性可分性 (Linear Separability): 单层感知机（使用阈值单元）只能解决线性可分的问题。

XOR 问题 (The XOR Problem):

- 逻辑异或 (XOR) 是典型的非线性可分问题。
- 输入 $(0, 0), (1, 1) \rightarrow 0$; 输入 $(0, 1), (1, 0) \rightarrow 1$ 。
- 在二维平面上无法画出一条直线将这两类分开。

结论: 这导致了单层感知机的衰落，必须引入多层网络和非线性单元（如 Sigmoid）。

3.1.2 梯度下降的目标 (Objective of Gradient Descent)

目的: 搜索假设空间，找到使误差最小的权重向量。

误差函数 (Loss Function): 使用**误差平方和 (Sum Squared Error)**。

$$E(w) = \frac{1}{2} \sum_e (y_e - o_e)^2$$

- y_e : 目标值 (Target)
- o_e : 实际输出值 (Output)
- $\frac{1}{2}$: 为了求导时抵消常数 2。

几何理解 (Geometry):

- **线性单元的误差曲面:** 是一个抛物面 (Parabolic bowl)，拥有唯一的全局最小值 (Single global minimum)。
- **移动策略:** 沿着误差曲面下降最快（最陡峭）的方向移动。
- **梯度方向:** 梯度的反方向即为下降方向。

3.2 第二板块：核心公式与推导 (Formulas & Derivations)

这部分是“重灾区”，必须精准记忆公式，特别是符号。

3.2.1 线性单元 (Linear Unit) 的梯度下降

激活函数: $o = \vec{w} \cdot \vec{x}$ (无阈值, 直接输出线性加权和)。

梯度推导 (Derivation):

- $\frac{\partial E}{\partial w_i} = \sum (y - o) \cdot \frac{\partial (y - o)}{\partial w_i}$
- $\frac{\partial E}{\partial w_i} = \sum (y - o)(-x_i)$
- $\frac{\partial E}{\partial w_i} = - \sum (y - o)x_i$

最终更新规则 (Update Rule):

$$w_i \leftarrow w_i + \eta \sum (y_e - o_e)x_{ie}$$

注意: 这里是加号 (+)。虽然梯度下降是 $w - \text{gradient}$, 但梯度本身带有负号, 负负得正。

3.2.2 Sigmoid 单元 (Sigmoid Unit) 的梯度下降

激活函数: $o = \sigma(S) = \frac{1}{1+e^{-S}}$, 其中 $S = \vec{w} \cdot \vec{x}$ 。

重要导数性质 (必须背):

$$\sigma'(S) = \sigma(S)(1 - \sigma(S))$$

即导数等于: 输出值 $\times$ (1 - 输出值)。

最终更新规则 (Update Rule):

$$w_i \leftarrow w_i + \eta \sum (y_e - o_e) \cdot o_e(1 - o_e) \cdot x_{ie}$$

对比: 比线性单元多了一项 $o_e(1 - o_e)$ 。

3.3 第三板块: 算法流程与计算套路 (Algorithm & Calculation)

这是考试中最容易出现的计算大题 (Hand Simulation), 请务必区分清楚两种模式。

3.3.1 批量梯度下降 (Batch Gradient Descent) / 标准梯度下降

逻辑: 看了所有样本, 算完总账, 才改一次权重。

计算步骤 (手算题解法):

1. 初始化 $\Delta w_i = 0$ (累加器清零)。

2. 遍历所有样本:

- 算出当前样本的输出 o 。
- 算出当前样本的修正量: $\delta = \eta(y - o)x$ (线性) 或 $\delta = \eta(y - o)o(1 - o)x$ (Sigmoid)。
- 将 δ 加入累加器: $\Delta w_i \leftarrow \Delta w_i + \delta$ 。

3. 循环结束后: 更新权重 $w_i \leftarrow w_i + \Delta w_i$ 。

路径: 平滑直线, 直接指向最小值。

3.3.2 增量/随机梯度下降 (Incremental / Stochastic Gradient Descent — SGD)

逻辑: 看一个样本, 改一次权重。看下一个样本, 用新权重再算。

计算步骤 (手算题解法):

1. 样本 1:

- 用当前 w 算输出 o 。
- 直接计算并更新 w : $w_i \leftarrow w_i + \eta(y - o)x_i$ 。

2. 样本 2:

- **关键:** 使用更新后的 w 来计算新的输出 o 。
- 再次更新 w 。

路径: Z字形 (Zig-zag), 围绕最速下降方向震荡前进, 但收敛速度通常更快, 且能跳出局部极小值。

3.3.3 手算题常见坑点 (Common Pitfalls)

- **偏置项 (Bias w_0):** 输入 x_0 通常默认为 1 (或 -1, 看题目设定, 课件例子中 x 输入向量通常包含 1)。不要漏算 w_0 的更新!
- **Sigmoid 计算:** 考试如果考 Sigmoid 手算, 记得带计算器算 e^{-x} , 并且千万别忘了乘 $o(1 - o)$ 。
- **正负号:** 检查公式是 $(y - o)$ 还是 $(o - y)$ 。如果是 $(y - o)$, 前面是加号; 如果是 $(o - y)$, 前面是减号。课件采用的是 $w + \eta(y - o)x$ 。

3.4 第四板块: 理论对比总结 (Comparison)

简答题或配对题考点。

特性	感知机规则	梯度下降
单元类型	阈值单元 (Step function)	线性单元 / Sigmoid (连续可导)
收敛条件	数据必须线性可分	数据不必线性可分
优化目标	最小化分类错误数	最小化误差平方和 $E(w)$
终止保证	线性可分时有限次收敛	渐进收敛 (Asymptotic), 不保证完全停止
结果	全对分类 (如果可分)	误差最小化 (允许存在少量分类错误)

3.5 考前 Checklist

上考场前最后看一眼:

- **公式记忆:** 能默写出线性单元和 Sigmoid 单元的 Δw 更新公式吗?
- **Sigmoid 导数:** 是否记住了 $\sigma' = \sigma(1 - \sigma)$?
- **算法区别:** 题目让你用 Batch 还是 Incremental? Batch 是最后加, Incremental 是步步改。
- **计算:** 遇到 $(y - o)$ 时, 如果 $y = 1, o = 0.8$, 误差是正的, 权重会增加 (假设输入正); 如果 $y = 0, o = 0.8$, 误差是负的, 权重会减小。这个直觉要有。
- **XOR:** 知道 XOR 是什么, 以及为什么单层感知机处理不了它 (因为画不出直线)。

4 Lecture 4: 多层感知机 (MLP) 与反向传播 (BP)

4.1 第一模块: MLP 基础架构与核心组件

考察重点: 激活函数的性质、为什么需要多层、网络的拓扑结构。

4.1.1 MLP 的核心定义

结构: 由多层感知机组成的分层结构 (输入层 - i 隐藏层 - h 输出层)。

必须是非线性的: 如果只是线性单元的级联 (多层线性叠加), 产生的仍然只是线性映射。必须使用非线性激活函数才能解决复杂问题 (如异或 XOR 问题)。

全连接 (Fully Connected): 前一层的每个节点都连接到下一层的所有节点。

前馈 (Feedforward): 信号单向传播 (前向), 没有环路。

4.1.2 激活函数 (Activation Functions)

训练算法要求激活函数必须具备三个性质: **可微 (Differentiable)**、**连续 (Continuous)**、**非线性 (Nonlinear)**。

Sigmoid 函数 (最常考):

- 公式:

$$o = \sigma(s) = \frac{1}{1 + e^{-s}}$$

- **导数特性:** $\sigma'(s) = o(1 - o)$ (反向传播计算的核心)。
- 输出范围: $(0, 1)$ 。

双曲正切 (Tanh):

- 公式:

$$o = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$

- 输出范围: $(-1, 1)$ 。
- 优势: 均值为 0, 有时收敛更快。

4.1.3 表示能力 (Representation Power)

- **两层网络 (1个隐藏层):** 理论上可以以任意精度逼近任何连续函数 (通用近似定理)。
- **布尔函数:** 可以精确学习所有布尔函数, 但隐藏单元数量可能随输入维度呈指数级增长。

- **隐藏层层数**：增加层数并不一定增加表示能力（对于近似连续函数，1层隐藏层足够）。

4.2 第二模块：反向传播算法 (Backpropagation) — 考试绝对核心

考察重点：完整的计算流程、梯度下降原理、关键公式推导（尤其是 δ/β 的计算）。

4.2.1 算法核心逻辑

目标函数 (Cost Function)：最小化均方误差 (MSE)。

$$E = \frac{1}{2} \sum_k (y_k - o_k)^2$$

优化方法：梯度下降 (Gradient Descent)。沿着误差梯度的反方向调整权重。

$$\Delta w = -\eta \frac{\partial E}{\partial w}$$

4.2.2 完整的计算步骤 (必须要背下来的算法流程)

1. **初始化**：权重设为小的随机值。
2. **前向传播 (Forward Pass)**:
 - 计算每一层的加权和 $s = \sum wx$ 。
 - 计算激活输出 $o = \sigma(s)$ 。
 - 从输入层 $\rightarrow$ 隐藏层 $\rightarrow$ 输出层。
3. **后向传播 (Backward Pass)**:
 - 计算输出层的误差项（收益 β_k ）。
 - 计算隐藏层的误差项（收益 β_j ）。
 - 从输出层 $\rightarrow$ 隐藏层。
4. **权重更新 (Weight Update)**:
 - 利用计算出的误差项修正权重。

4.2.3 核心计算公式 (死记硬背区)

这里使用 β (有时也叫 δ) 表示误差项。假设激活函数是 Sigmoid。

A. 输出层节点 k 的误差项 β_k :

$$\beta_k = \underbrace{(y_k - o_k)}_{\text{误差}} \cdot \underbrace{o_k(1 - o_k)}_{\text{导数}}$$

B. 隐藏层到输出层权重 w_{jk} 的更新量:

$$\Delta w_{jk} = \eta \cdot \beta_k \cdot o_j$$

(学习率 $\times$ 输出层误差 $\times$ 隐藏层输入)

C. 隐藏层节点 j 的误差项 β_j (难点):

$$\beta_j = \underbrace{o_j(1 - o_j)}_{\text{自身导数}} \cdot \left[\sum_k \beta_k w_{jk} \right]$$

(自身导数 $\times$ 下游加权误差之和)

D. 输入层到隐藏层权重 w_{ij} 的更新量:

$$\Delta w_{ij} = \eta \cdot \beta_j \cdot o_i$$

4.3 第三模块：训练策略与调优 (Hyperparameters & Tuning)

考察重点：如何让网络收敛、动量的作用、初始化与停止条件。

4.3.1 训练模式

在线学习 (On-line / Incremental): 每输入一个样本，就更新一次权重。

- 优点：随机性有助于跳出局部极小值，收敛通常更快。

批量学习 (Batch Learning): 输入所有样本，计算总梯度后，才更新一次权重。

- 特点：理论上梯度方向更准，但实际效果常不如在线学习。

样本顺序：随机顺序 (Random presentation) 优于固定顺序 (Sequential)，能避免循环陷阱。

4.3.2 关键参数设置

权重初始化：必须是随机的小数值 (如 -0.5 到 0.5)。

- **禁忌：** 不能全部初始化为0或相同的值 (会导致对称性，网络无法学习)。

隐藏节点数量:

- 经验公式: $n = (\text{inputs} + \text{outputs})/2$ 。
- 太少 $\rightarrow$ 欠拟合; 太多 $\rightarrow$ 过拟合且慢。

学习率 (η):

- 典型值: $0.05 < \eta < 0.75$ 。
- 太小 $\rightarrow$ 收敛极慢。
- 太大 $\rightarrow$ 震荡 (Oscillation), 无法收敛。

4.3.3 动量项 (Momentum) — 必考优化技巧

目的: 稳定权重变化, 加速收敛, 跳出局部极小值。

物理意义: 给权重更新增加“惯性”。

公式:

$$\Delta w(t) = -\eta \frac{\partial E}{\partial w(t)} + \alpha \Delta w(t-1)$$

系数 α : 典型值 0.6 - 0.9。

- $\alpha = 0$: 普通 BP 算法。
- $\alpha = 1$: 完全忽略当前梯度 (只看历史)。

4.4 第四模块: 过拟合与泛化 (Overfitting & Generalization)

考察重点: 过拟合的表现、原因、以及三种具体的解决方法。

4.4.1 现象与原因

现象: 训练误差很低, 但测试/验证误差很高 (泛化能力差)。模型“死记硬背”了数据。

原因:

- 训练数据包含噪声 (Noise)。
- 模型太复杂 (参数过多, 多于样本数)。

4.4.2 解决方案 (三大法宝)

早停法 (Early Stopping):

- 使用验证集 (Validation Set)。

- 当训练误差下降但验证误差开始上升时，立即停止训练。
- 保留验证误差最低时的权重。

正则化 / 权重衰减 (Weight Decay):

- 在误差函数中加入惩罚项，限制权重的大小。
- 原理：避免大权重导致的函数剧烈震荡（输出饱和、梯度消失）。

交叉验证 (Cross-Validation):

- **K-Fold**: 将数据分为 K 份，轮流做验证集。计算平均最佳 Epoch。
- **留一法 (LOOCV)**: $K = N$ (样本总数)。每次只留 1 个做验证。适用于极小数据集。

4.5 第五模块：MLP 的能力总结

优点： 能近似任意非线性函数，解决复杂分类问题。

缺点 (局限性)：

- 容易陷入**局部极小值 (Local Minima)**（不一定是全局最优）。
- 对初始权重非常敏感。
- 是“黑盒”，难以解释内部权重含义。
- 训练时间长（需要成百上千个 Epoch）。

4.6 考前最后叮嘱

- **公式别写错**：尤其是 β 的计算，分清楚谁是输出层，谁是隐藏层。隐藏层的 β 包含求和符号 $\sum$ 。
- **概念别混淆**：分清 Epoch (一轮所有样本) 和 Iteration (一次更新)。分清 Batch 和 Online。
- **过拟合图示**：脑子里要有那个“训练误差下降，验证误差先降后升”的 U 型曲线图。
- **计算器**：如果考试允许，遇到简单的 BP 计算题，按计算器时注意 Sigmoid 的计算顺序。

5 Lecture 5: 卷积神经网络 (CNN) 最终冲刺

5.1 第一板块：核心架构理念 (Core Philosophy)

这部分是简答题的常客，必须准确背出关键词。

5.1.1 端到端学习 (End-to-end Learning)

定义： 从原始像素直接映射到最终标签，不再人为区分特征提取和分类器。

优势： 系统所有部分都是自适应的，能够根据具体任务自动优化特征。

5.1.2 特征层级结构 (Feature Hierarchy)

层级逻辑： 第一层识别边缘 (Edges)，第二层组合成物体部件 (Object parts)，第三层形成完整物体 (Objects)。

本质： 通过函数的复合 (Function Composition) 来构建复杂的非线性系统。

5.1.3 CNN 的三大核心思想 (Key Ideas) [高频考点]

1. **局部感受野 (Local Receptive Fields):** 神经元仅连接前一层的一个子集（局部区域），专注于提取空间局部特征。
2. **权值共享 (Shared Weights):** 同一个特征图上的所有单元共享相同的权重向量和偏置，这极大减少了自由参数的数量并提供了**平移不变性**。
3. **空间/时间下采样 (Sub-sampling):** 即池化操作，用于降低空间分辨率，减少对位移和扭曲的敏感度。

5.2 第二板块：核心数学公式与计算 (Formulas & Calculations)

这是计算大题的“生死线”，绝不能算错。

5.2.1 卷积输出尺寸计算 [必背]

基础公式： 若输入为 $N \times N$ ，滤波器为 $m \times m$ ，则输出尺寸为：

$$(N - m + 1) \times (N - m + 1)$$

5.2.2 卷积层计算逻辑

求和公式：

$$x_{ij}^l = \sum_{a=0}^{m-1} \sum_{b=0}^{m-1} \omega_{ab} y_{(i+a)(j+b)}^{l-1}$$

考点： 记住这是局部区域的加权求和，再加上偏置项后经过非线性激活。

5.2.3 激活函数 (Activation Functions)

ReLU:

$$f(x) = \max(0, x)$$

Softmax:

$$\hat{y}_i = \frac{e^{w_{3i}h_2 + b_{3i}}}{\sum_k e^{w_{3k}h_2 + b_{3k}}}$$

用于计算属于某个预定义类别的概率。

5.2.4 优化算法更新 (SGD)

基本更新:

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

带动量更新:

$$\begin{aligned}\Delta &\leftarrow 0.9\Delta + \frac{\partial L}{\partial \theta} \\ \theta &\leftarrow \theta - \eta\Delta\end{aligned}$$

5.3 第三板块：关键组件对比与选型逻辑 (Analysis & Comparison)

5.3.1 ReLU vs. Sigmoid (为什么选 ReLU?) [高频考点]

- 计算效率： ReLU 仅涉及简单的 if 判断，而 Sigmoid 需要计算复杂的指数。
- 梯度消失问题： Sigmoid 在输入较大或较小时易饱和，导致梯度接近 0；ReLU 在 $x > 0$ 时不饱和，能有效缓解梯度消失，适合深层网络。

5.3.2 池化操作 (Pooling)

最大池化 (Max Pooling): 提取区域内最强信号，提供平移不变性并减少计算量。

平均池化 (Average Pooling): 平滑特征，但可能导致高亮特征被削弱（对比度降低）。

5.3.3 归一化 (Normalization)

局部对比度归一化 (LCN): 通过减去均值并除以标准差，使特征图的纹理细节更锐利，有助于特征提取。

5.4 第四板块：训练与进阶策略 (Training & Advanced)

5.4.1 反向传播 (Back-propagation) [核心考点]

本质： 利用链式法则，从 Loss 层向输入层反向计算损失对每个参数的梯度。

前提： 所有的中间层函数必须是可微的 (Differentiable)。

5.4.2 迁移学习 (Transfer Learning) [高频考点]

定义： 将在一个任务中学到的知识（通常是卷积层参数）应用到新的任务中。

做法： 复用在大规模数据集（如 ImageNet）上预训练好的参数，用于解决目标任务数据不足的问题。

逻辑： 底层卷积提取的是通用特征（边缘、形状），可以跨任务复用。

5.5 考前寄语

这份清单涵盖了本课件 95% 以上的考点。建议你闭上眼睛尝试推导一遍卷积尺寸公式和反向传播的梯度流动方向。

6 Lecture 6: 径向基函数网络 (RBFN) 与聚类分析

这是为你定制的考试特化知识点清单。

6.1 第一板块: RBFN 核心理论与设计直觉 (Concept & Rationale)

这一部分主要应对概念选择题和简答题, 解释“为什么要用 RBF”。

6.1.1 基本定义

RBFN 的本质是将神经网络设计视为**曲线拟合 (Curve-fitting)** 或**函数逼近 (Function approximation)** 问题。

目标是寻找高维空间中的插值曲面 (Interpolating surface)。

6.1.2 核心直觉 (The Rationale)

非线性映射 (Non-linear mapping): 将输入样本投射到高维隐藏空间。

线性可分性: 依据 Cover 定理思想, 样本在高维空间比在低维空间更容易被**线性分离 (Linearly separable)**。

两步走策略:

- 输入层 $\rightarrow$ 隐藏层: **非线性**变换 (把问题变简单)。
- 隐藏层 $\rightarrow$ 输出层: **线性**变换 (快速求解)。

6.1.3 正则化网络 vs. RBF 网络 (必须分清)

正则化网络 (Regularization Network):

- 中心点数量 = 样本数量 ($N \text{ centers} = N \text{ samples}$)。
- 是万能逼近器 (Universal Approximator)。
- 缺点: 计算效率极低, 需要对巨大的矩阵求逆, 容易过拟合。

RBF 网络 (Standard RBFN):

- 中心点数量远小于样本数量 ($M < N$)。
- 目的: 通过缩减网络拓扑结构, 在减少计算量的同时逼近正则化网络的解。

6.2 第二板块: 网络架构与数学模型 (Architecture & Math)

这一部分涉及具体的函数形式, 是计算题的基础。

6.2.1 三层网络结构

Input Layer (输入层): 仅传递向量，不计算。

Hidden Layer (隐藏层): 执行非线性变换。

- 使用径向基函数 $\phi(\|\mathbf{x} - \mathbf{t}_i\|)$ 。
- 高斯函数 (Gaussian) 是最常用的基函数:

$$\phi(x) = \exp\left(-\frac{\|x - t\|^2}{2\sigma^2}\right)$$

- t : 中心 (Center)。
- σ : 宽度/方差 (Variance)。
- 特性: 局部响应 (Local response), 随着距离增加单调递减。

Output Layer (输出层): 执行线性变换 (加权求和)。

6.2.2 网络输出公式 (单样本)

$$F(x) = \sum_{i=1}^M w_i \phi_i(\|x - t_i\|)$$

- M : 基函数/隐藏层神经元的数量。
- w_i : 连接隐藏层和输出层的权重。

6.3 第三板块: RBF 学习算法与权重计算 (The “Must-Memorize” Formulas)

这是考试中计算题的核心，设计矩阵和伪逆公式必须刻在脑子里。

6.3.1 训练目标

找到最佳权重 W ，使得网络输出 $o(x_i)$ 等于期望输出 d_i 。

6.3.2 设计矩阵 (Design Matrix) Φ

这是一个 $N \times M$ 的矩阵 (N 个样本, M 个中心)。

元素 ϕ_{ji} 代表第 j 个样本在第 i 个隐藏神经元上的激活值。

构造形式:

$$\Phi = \begin{bmatrix} \phi_1(\|x_1 - t_1\|) & \dots & \phi_M(\|x_1 - t_M\|) \\ \vdots & \ddots & \vdots \\ \phi_1(\|x_N - t_1\|) & \dots & \phi_M(\|x_N - t_M\|) \end{bmatrix}$$

6.3.3 权重求解公式 (The Pseudo-Inverse Method) [必背]

线性方程组: $\Phi W = d$

由于 Φ 不是方阵, 无法直接求逆, 使用伪逆求解:

$$\mathbf{W} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{d}$$

注意: 这个公式是一步到位的 (One-shot), 不需要像 BP 网络那样迭代训练权重。

6.3.4 RBF 完整训练流程

1. 初始化: 确定中心数量 M , 确定中心位置 t_i (用 K-means), 确定方差 σ_i 。
2. 计算 Φ : 将所有样本代入高斯函数计算设计矩阵。
3. 计算 W : 代入伪逆公式求出权重。

6.4 第四板块: 中心点选择与 K-means 聚类 (Center Selection & Algorithms)

既然 RBF 需要 $M < N$ 个中心, 如何选这 M 个点? **重点考察 K-means。**

6.4.1 选择方法

随机选择 (Random): 简单, 适用于样本分布均匀且有代表性的情况。

K-means 聚类: 最常用的方法, 寻找数据的高密度区域作为中心。

6.4.2 K-means 算法流程 (闭眼都要能背出步骤)

1. **Step 1: Initialization** — 选择 K 个簇, 随机初始化质心位置。
2. **Step 2: Assignment** — 计算每个样本到所有质心的距离, 将其分配给最近的那个质心。
3. **Step 3: Update** — 重新计算质心位置 (取当前簇内所有点的均值 Mean)。
4. **Step 4: Termination** — 检查是否收敛 (Converge)。如果没有变化或误差最小化, 则停止; 否则回到 Step 2。

6.4.3 距离度量

通常使用欧几里得距离 (Euclidean distance, $q = 2$)。

$$d(i, j) = \sqrt{\sum |x_{ip} - x_{jp}|^2}$$

6.4.4 K-means 的缺陷 (Issues) [简答题高频考点]

- **必须预设 K 值**：事先不知道分几类最合适。
- **不适合非凸形状 (Non-convex shapes)**：例如环形、笑脸形数据，K-means 会切断它们（因为它基于距离，倾向于球形簇）。
- **对噪声/异常值敏感 (Sensitive to noise/outliers)**：离群点会严重拉偏均值。

改进方法：

- **Repeated k-means**：多次随机初始化，取误差最小的那次结果。

6.5 第五板块：概念大对决 RBF vs MLP (Comparisons)

这是考试中最后一道压轴大题最喜欢考的“对比题”。

比较维度	RBF Networks	MLP
隐藏层数量	仅一层 (One hidden layer)	一层或多层
逼近性质	局部逼近 (Local approximation)	全局逼近 (Global approximation)
激活函数	节点间不同 (Different)	层内节点相同 (Same)
非线性位置	仅隐藏层是非线性的	隐藏层和输出层均是非线性的
计算机制	基于欧几里得距离	基于内积 (Inner products)
训练速度	快（通常一步计算权重）	慢（需要反向传播迭代）

6.6 考前最后 Check

- **公式默写**： $W = (\Phi^T \Phi)^{-1} \Phi^T d$ 。
- **图形判断**：如果试卷给出一个“双环”或者“月牙形”数据图，问 K-means 能不能聚类？答案是**不能** (Non-convex)。
- **流程复述**：能不能口述出 K-means 的 Update 步骤是干什么？（算均值）。

7 Lecture 7: 时间序列与 Elman 网络 (Time Series & Elman Network)

7.1 第一板块：时间序列基础理论 (Time Series Fundamentals)

这部分主要考察定义题和判断题，重点在于“平稳性”。

7.1.1 核心定义 (Key Definition)

时间序列：一系列按时间顺序 (timely ordered) 排列的观测值 y_t 。

关键约束：必须是在等距的时间间隔 (equidistant time intervals) 内记录的。

符号： y_t 表示第 t 个周期的观测值 ($t = 1, 2, \dots, n$)。

7.1.2 统计属性与平稳性 (Statistical Properties & WSS)

预测的前提：连续时间点的值之间必须存在相关性 (Correlation)，否则无法预测。

WSS (广义平稳 / Wide-sense stationarity): 必考概念

一个随机过程被称为 WSS，必须满足两个条件：

- 均值 (Mean) 和自协方差 (Autocovariance) 不随时间变化 (即常数)。
- 方差 (Variance) 在所有时间点上都是有限的 (finite)。

理解：如果统计特性一直变 (非平稳)，模型就无法从过去的数据中学到稳定的规律。

7.2 第二板块：预测问题的数学建模 (Prediction Problem Formulation)

这部分考察如何将实际问题转化为数学函数问题。

7.2.1 预测公式

目标：这是一个函数逼近 (Function Approximation) 问题。

公式：

$$\hat{x}[t+s] = f(x[t], x[t-1], \dots)$$

- **输入：**过去的时间序列 $\{x[t], x[t-1], \dots\}$ 。
- **输出：**未来时刻的估计值 $\hat{x}[t+s]$ 。
- **s ：**预测视野 (Horizon of prediction)。最简单的情况是单步预测 ($s = 1$)。

7.2.2 建模三部曲 (Standard Procedure)

1. **Step 1:** 假设一个生成模型 (Generative model)。
2. **Step 2:** 训练 (Train)。
 - Input: t 时刻之前的序列。
 - Target: t 时刻之后的实际值。
3. **Step 3:** 运行 (Run)。使用训练好的模型预测 $\hat{x}[t + s]$ 。

7.2.3 预测的难点 (Difficulties)

- **非平稳性 (Non-stationarity):** 数据的基本面随时间改变 (最难处理)。
- **噪声 (Noise):** 数据中包含错误点。
- **数据有限:** 训练集/测试集划分困难。

7.3 第三板块: 静态网络与滑动窗口 (Static Networks: TDNN)

这部分考察旧方法的局限性以及如何构建输入向量。

7.3.1 滑动窗口机制 (Sliding Window)

原理: 将时间序列问题强行转化为静态映射问题。

输入构建:

- 选取窗口大小 s 。
- 输入向量: $\{X_{t-s}, \dots, X_{t-1}, X_t\}$ 。

考试技巧: 看到题目给出一串数字让构建输入, 就是截取一段作为 Input, 下一个数字作为 Target。

归一化: 通常将数据映射到 $[0, 1]$ 区间作为激活值。

7.3.2 时间延迟网络 (Time Delayed Networks, TDNN)

本质: 带有外部缓冲 (External Buffer) 的前馈网络。

记忆类型: 隐式记忆 (Implicit Memory)。它通过空间 (增加输入节点) 来换取时间信息。

局限性: 网络内部没有状态, 它依然是一个静态网络 (Static Network)。

7.4 第四板块：动态神经网络基础 (Dynamical/Recurrent Networks)

这部分考察为什么需要 RNN 以及 RNN 的核心特征。

7.4.1 静态 vs 动态 (Static vs Dynamic)

静态网络 (MLP/RBF): 学习 Single Input $\rightarrow$ Single Output 的映射。

动态网络 (RNN): 学习 Single Input $\rightarrow$ Sequence of Responses 的映射。

关键: 动态网络不仅基于输入运行，还基于状态 (State) 运行。

7.4.2 核心特征

- **连接循环 (Connection Cycle):** 至少有一个环。
- **回响 (Reverberate):** 即使没有输入，激活值也能在网络中持续存在。
- **工作记忆 (Working Memory):** 利用这种回响能力来维持短期记忆。

7.5 第五板块：Elman 网络 (The Elman Network)

这是本章最重要的核心考点，包含架构图、公式和运作机制。

7.5.1 定义与归类

提出者: Jeffrey L. Elman (1990)。

类型: 简单循环神经网络 (Simple Recurrent Neural Network, SRNN)。

7.5.2 四层架构 (Architecture)

Input Layer: 接收当前时刻输入 $x(t)$ 。

Hidden Layer: 形成内部表征，计算核心。

Output Layer: 线性组合输出结果。

Context Layer (承接层/上下文层) [关键]:

- **作用:** 充当短期记忆。
- **机制:** 存储上一时刻 $(t - 1)$ 隐藏层神经元的输出值。
- **连接权重:** 从 Hidden Layer $\rightarrow$ Context Layer 的权重固定为 1 (Fixed at 1)。这本质上是精确复制 (Copied exactly)。
- **反馈:** Context Layer 的值作为额外的输入信号，通过权重 U 反馈回 Hidden Layer。

7.5.3 数学方程 (Equations) [必须背诵]

输出层: $y_k(t) = g(net_k)$ (与普通 MLP 相同)。

隐藏层 (核心公式):

$$net_j(t) = \underbrace{\sum_i v_{ji}x_i(t)}_{\text{当前输入}} + \underbrace{\sum_h u_{jh}y_h(t-1)}_{\text{历史记忆 (来自 Context)}} + \theta_j$$

- $x_i(t)$: 当前输入。
- $y_h(t-1)$: 上一时刻隐藏层的输出 (即 Context Layer 的值)。
- u_{jh} : 延迟反馈权重。

7.5.4 训练算法

算法: 标准的 BP (Back Propagation) 算法。

误差函数: 均方误差 (Sum of Squared Errors)。

$$E = \sum [y(k) - d(k)]^2$$

7.6 考前自测问题 (Self-Check)

在上考场前, 请确保你能口述回答以下问题:

- **判定:** 什么样的随机过程是 WSS (广义平稳)? (答: 均值/自协方差不变, 方差有限)。
- **操作:** 给定一个时间序列, 如何构建滑动窗口为 3 的 MLP 训练样本?
(答: Input $[x_{t-2}, x_{t-1}, x_t]$, Target x_{t+1})。
- **机理:** Elman 网络中的 Context Layer 具体存了什么? 它的输入权重是多少?
(答: 存了 $t-1$ 时刻的隐藏层输出, 权重固定为 1)。
- **对比:** 为什么 TDNN 被称为“隐式”记忆, 而 Elman 是“显式”处理序列? (答: TDNN 靠外部排队, Elman 靠内部循环)。

8 Lecture 8: 循环神经网络与 LSTM (RNN & LSTM)

8.1 第一板块: RNN 的核心逻辑与应用场景 (The “Why” & “What”)

这部分通常出现在选择题或简答题的第一问，考察你对网络类型匹配任务的理解。

8.1.1 核心动机 (Motivation)

前馈网络 (Feed Forward) 的局限： 输入输出大小固定，无法处理序列，假设样本独立。

RNN 的优势：

- 处理序列数据 (Sequences)。
- 利用上下文信息 (Context/History): t 时刻的输出依赖于 t 时刻的输入 + $t-1$ 时刻的记忆。

关键假设： 对序列中的每个元素执行相同的任务（参数共享）。

8.1.2 五大架构类型与考题映射 (Architectures & Examples)

考试中给出一个场景，让你选架构类型，**必须背熟**：

- **One-to-One (Vanilla):** 非 RNN，图像分类。
- **One-to-Many:** 非序列输入 $\rightarrow$ 序列输出。例子：看图说话 (Image Captioning)。
- **Many-to-One:** 序列输入 $\rightarrow$ 单一输出。例子：情感分析 (Sentiment Analysis)。
- **Many-to-Many (Async):** 序列输入 $\rightarrow$ 序列输出（先读完再输出）。例子：机器翻译 (Machine Translation — 英译法)。
- **Many-to-Many (Synced):** 序列输入 $\rightarrow$ 序列输出（每步同步）。例子：视频帧分类 (Video Classification)。

8.1.3 拓扑结构概念

Partly Recurrent: 循环仅限制在隐藏层。

Fully Recurrent: 每个节点都接收所有其他节点的输入（全连接）。

8.2 第二板块: RNN 数学原理与训练 (The Math & Algorithm)

这部分涉及公式填空或计算推导，必须记准符号。

8.2.1 RNN 前向传播公式 (Forward Pass)

记号说明: U (输入权重), W (循环权重), V (输出权重)。

Step 1: 隐藏层输入 (融合当前与过去)

$$a_h(t) = Ux(t) + Ws(t-1)$$

Step 2: 隐藏层状态/记忆 (激活)

$$s(t) = f_h(a_h(t))$$

Step 3: 输出层预激活

$$a_o(t) = Vs(t)$$

Step 4: 最终输出

$$o(t) = f_o(a_o(t))$$

8.2.2 损失函数 (Loss Function)

总误差: 是所有时间步误差的累加 (Sum over time)。

$$E_{total} = \sum_{t=t_0}^{t_1} \frac{1}{2} (d(t) - o(t))^T (d(t) - o(t))$$

8.2.3 训练算法: BPTT (Back Propagation Through Time)

原理: 将 RNN 按时间展开 (Unfolding) 为一个深层前馈网络。

核心特征:

- 权重 (U, V, W) 在所有时间步中是复制/共享的。
- 梯度来源于每一个时间步的贡献。

关键依赖: Loss 不仅对当前层反向传播, 还通过 Hidden State 向上一时间步传播。

8.2.4 核心问题: 梯度消失 (Vanishing Gradients)

现象: 无法学习长距离依赖 (Long-term dependencies)。

数学原因:

- 反向传播时涉及多次导数连乘。
- Sigmoid/Tanh 导数 < 1 (Sigmoid 导数最大 0.25)。

- 随着层数（时间步）增加，梯度趋近于 0。

后果：网络前面的层（早期的记忆）无法获得有效的权重更新，导致“遗忘”。

8.3 第三板块：LSTM 深度解析 (The Solution)

这是本章最难、也是最重要的部分。必须死记公式和逻辑流程。

8.3.1 设计初衷

核心目的：解决梯度消失。

核心组件：CEC (Constant Error Carousel)。

- 是一个权重固定为 1 的自连接循环。
- 作用：保证误差流在反向传播时不衰减、不增长 (Constant)，从而保留长时记忆。

8.3.2 LSTM 细胞的三大信号流 (Gates Logic)

LSTM 通过“门”来控制信息流。门由 Sigmoid (σ) 激活，输出 0~1。

- 0: 禁止 (Denied/Close)。
- 1: 允许 (Granted/Open)。

8.3.3 LSTM 完整计算流程 [公式必背]

请按顺序记忆以下 5 步：

Step 1: 处理输入数据 (Input Processing)

决定哪些来自 $x(t)$ 和 $y(t-1)$ 的信息是候选记忆。

$$z(t) = g(W_z x(t) + R_z y(t-1))$$

(g 通常是 $\tanh$)

Step 2: 计算输入门 (Input Gate — Write Control)

决定允许多少 $z(t)$ 写入细胞。

$$i(t) = \sigma(W_{in} x(t) + R_{in} y(t-1))$$

Step 3: 更新细胞状态 (Cell State Update) [最核心公式]

新状态 = (新输入 $\times$ 输入门) + 旧状态

$$c(t) = z(t) \odot i(t) + c(t-1)$$

(注: $\odot$ 代表 Hadamard Product 逐元素乘积; 公式中直接加 $c(t-1)$ 体现了 CEC 特性)

Step 4: 计算输出门 (Output Gate — Read Control)

决定允许细胞内部状态 $c(t)$ 被读取多少。

$$o(t) = \sigma(W_{out}x(t) + R_{out}y(t-1))$$

Step 5: 计算最终输出 (Final Output)

先挤压细胞状态, 再通过输出门过滤。

$$y(t) = h(c(t)) \odot o(t)$$

(h 通常是 $\tanh$)

8.4 第四板块: LSTM 优缺点评估 (Critical Evaluation)

通常用于简答题最后一部分, 考察辩证思维。

8.4.1 优势 (Advantages)

- **解决梯度消失:** 通过 CEC 实现 Non-decaying error backpropagation。
- **长时记忆:** 能桥接输入和目标之间很长的时间滞后 (Time lags)。
- **鲁棒性:** 能处理噪声 (Noise) 和连续值 (Continuous values)。
- **无需精细调参:** 相比普通 RNN, 不需要费力调整参数来捕捉长距离信息。

8.4.2 局限性与风险 (Limitations)

- **线性增长 (Linear Growth):** 由公式 $c(t) = \dots + c(t-1)$ 可知, 如果输入流持续存在, 细胞状态 $c(t)$ 会线性增长。
- **饱和 (Saturation):** 当 $c(t)$ 无界增长变得非常大时, 输出挤压函数 $h(c(t))$ (即 $\tanh$) 会进入饱和区。
- **后果:** 梯度再次变为 0, 导致网络无法继续有效学习。

8.5 考前 1 分钟自测 (Checklist)

- 你能写出 RNN 的 hidden state 更新公式吗? ($s(t) = \dots$)
- 你能解释为什么 BPTT 会导致梯度消失? (关键字: Sigmoid 导数, 连乘)
- LSTM 中的 CEC 是什么? 它为什么能解决梯度消失? (关键字: Weight 1, Constant flow)
- 请写出 LSTM 细胞状态 $c(t)$ 的更新公式。(注意 $\odot$ 符号的使用)
- 如果考试问 LSTM 的缺点, 你要回答哪一点? (关键字: Linear growth, Saturation)

9 Lecture 9: 主成分分析 (Introduction to PCA)

9.1 第一板块: 数学基石 (Linear Algebra Foundations)

这是所有计算题的底层工具，如果这里算错，后面全错。

9.1.1 特征值与特征向量 (Eigenvalues & Eigenvectors)

核心定义: 对于方阵 A ，若存在非零向量 v 和标量 λ 满足：

$$Av = \lambda v$$

则 v 是特征向量， λ 是特征值。

计算套路 (必考):

1. 列出特征方程: $|A - \lambda I| = 0$ (行列式为0)。
2. 解方程求出所有 λ 。
3. 将每个 λ 代回 $(A - \lambda I)v = 0$ 解出对应的向量 v 。

注意: 考试中通常要求特征向量是单位化 (Normalized) 的，即 $\|v\| = 1$ 。

9.1.2 实对称矩阵 (Real Symmetric Matrices)

定义: $S = S^T$ (例如协方差矩阵就是实对称的)。

三大考试推论 (背诵):

- 实数性: 所有特征值 λ 必为实数 (Real)。
- 正交性: 不同特征值对应的特征向量相互正交 (Orthogonal) ($v_1 \cdot v_2 = 0$)。
- 非负性: 若矩阵是半正定的 (Positive Semidefinite)，则所有 $\lambda \geq 0$ 。

9.2 第二板块: 矩阵分解核心 (Matrix Decomposition)

理解 PCA 怎么拆解数据的核心。

9.2.1 特征分解 (Eigen Decomposition / Spectral Decomposition)

适用对象: 方阵 (Square Matrix)。

公式:

$$S = U\Lambda U^{-1}$$

对称矩阵特例 (重点): 若 S 是对称矩阵, 则 U 是正交矩阵 (Q), 公式变为:

$$S = Q\Lambda Q^T$$

- Q : 列向量是归一化且正交的特征向量。
- Λ : 对角线上是特征值 (从大到小排列)。

关键性质: $Q^{-1} = Q^T$ (计算逆矩阵时的捷径)。

9.2.2 奇异值分解 (SVD)

适用对象: 任意 $m \times n$ 矩阵 A 。

公式:

$$A = U\Sigma V^T$$

成分构成 (填空/计算题考点):

- U : AA^T 的特征向量 (正交)。
- V : $A^T A$ 的特征向量 (正交)。
- Σ : 对角线上是奇异值 σ 。

σ 与 λ 的关系:

$$\sigma_i = \sqrt{\lambda_i}$$

(其中 λ_i 是 $A^T A$ 的特征值)

9.3 第三板块: PCA 理论推导逻辑 (Theory & Derivation)

考试简答题或证明题的高频区。

9.3.1 核心目标

目标: 降维 (Reduce dimensionality)。

原则: 最大化数据的变异性/方差 (Maximize Variation/Variance)。

假设: 数据已中心化 ($E[x] = 0$)。

9.3.2 推导链条 (逻辑必须清晰)

定义投影: 将数据 x 投影到方向 q 上, 得到标量 $a = q^T x$ 。

定义方差:

$$\text{Variance} = E[a^2] - (E[a])^2$$

由于数据已中心化, $E[a] = 0$, 故方差 $\sigma^2 = E[a^2]$ 。

展开公式:

$$\sigma^2 = E[(q^T x)(x^T q)] = q^T E[xx^T]q = q^T Rq$$

(其中 R 是协方差矩阵)

优化问题:

- Maximize: $q^T Rq$
- Subject to: $q^T q = 1$ (单位向量约束)

结论:

- 利用拉格朗日乘数法可得 $Rq = \lambda q$ 。
- 方差最大化的方向 q 就是协方差矩阵 R 的最大特征值对应的特征向量。
- 最大方差的值 σ^2 等于该特征值 λ 。

9.4 第四板块: PCA 计算流程与算法 (Algorithm Logic)

这是面对一道 “Perform PCA on the following data” 大题时的标准操作手册。

9.4.1 步骤 1: 数据预处理

- 计算均值向量 μ 。
- 中心化 (Centering): $x_{new} = x_{old} - \mu$ 。

9.4.2 步骤 2: 构建协方差矩阵

计算 $R = \frac{1}{N-1}XX^T$ (或 $E[xx^T]$)。

注意: 如果题目给的是样本矩阵, 注意是除以 N 还是 $N-1$, 通常理论推导用 $E[\dots]$, 计算题看题目约定, 大多数情况简单计算直接用 XX^T 的结构分析特征。

9.4.3 步骤 3: 特征值分解

- 求解 R 的特征方程 $|R - \lambda I| = 0$ 。
- 得到特征值 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ 。
- 求出对应的单位特征向量 $u_1, u_2, \dots, u_n$ 。

9.4.4 步骤 4: 选择主成分 (降维)

- 若要降到 K 维, 选择前 K 个最大的 λ 对应的向量 $u_1, \dots, u_K$ 。

- 构成变换矩阵 $U_{reduced} = [u_1, \dots, u_K]$ 。

9.4.5 步骤 5: 投影 (计算新坐标)

新坐标 Y (或 $\hat{x}$) 计算公式:

$$Y = U_{reduced}^T X$$

物理意义: 原始数据在主成分方向上的投影长度。

9.5 第五板块: 批判性分析 (Critical Analysis)

用于回答 “Discussion” 或 “True/False” 类型的题目。

9.5.1 局限性 (Limitations)

- **线性限制:** PCA 只能处理线性关系。对于非线性流形 (如瑞士卷形状), 效果极差。
- **可解释性差:** 主成分是原始特征的混合 (线性组合), 通常失去了物理意义, 难以解释。
- **方差假设:** PCA 假设 “方差小 = 噪声/不重要”。但这不总是对的, 有时候小方差包含关键分类信息。

9.5.2 特征脸 (Eigenfaces) 案例

- **本质:** 图像 -j 高维向量。
- **操作:** 对人脸图像集的协方差矩阵做分解。
- **结果:** 特征向量可以被还原为图像 (看起来像鬼脸), 称为 Eigenfaces。
- **应用:** 人脸识别、图像压缩 (用少量系数就能重构人脸)。

9.6 考前叮嘱 (Final Tips)

- **计算器:** 确保你会用计算器解一元二次方程 (求特征值时)。
- **正负号:** 特征向量的方向是双向的 (v 和 $-v$ 都是正确的特征向量), 如果你算出的符号和标准答案相反, 通常也是对的, 但要保持整个向量符号一致。
- **单位化:** 只要涉及到投影矩阵 Q 或 U , 务必记得把特征向量除以它的模长 $\sqrt{x^2 + y^2}$ 。
- **维度检查:** X 是 $N \times 1$, U^T 是 $K \times N$, 乘完变成 $K \times 1$ 。做题时时刻检查矩阵维度是否匹配。

10 Lecture 10: 无监督学习 (Unsupervised Learning)

核心逻辑线索：本章围绕“如何在没有标签的情况下发现数据结构”展开。

- 路径 1 (特征提取) : Hebbian $\rightarrow$ Oja's Rule $\rightarrow$ PCA $\rightarrow$ Auto-encoders。
- 路径 2 (聚类与分类) : K-means $\rightarrow$ Competitive Learning (WTA) $\rightarrow$ SOM (基础)。

10.1 第一板块：赫布学习与 Oja 规则 (基础与修正)

这部分是神经网络权值更新的数学基础，必考公式和推导逻辑。

10.1.1 赫布学习 (Hebbian Learning)

核心原理：“Cells that fire together, wire together.” (一起激发的神经元，连在一起)。

数学公式 (必记):

$$\Delta w_{ji} = \eta y_j x_i$$

- y_j : 后突触输出
- x_i : 前突触输入
- η : 学习率

几何意义：权重向量 $\mathbf{w}$ 会向输入向量 $\mathbf{x}$ 的方向旋转 (基于点积/相似度)。

致命缺陷 (考点): 不稳定性 (Instability)。

- 权重会无界增长 (Increase without bounds)。
- 正的更正，负的更负。
- 原因：公式中缺乏限制项。

10.1.2 Oja 规则 (Oja's Rule)

目的：解决赫布学习的“无界增长”问题，实现归一化 (Normalization)。

近似公式 (必记):

$$w_{ji}(n+1) = w_{ji}(n) + \eta y_j(n)[x_i(n) - y_j(n)w_{ji}(n)]$$

关键组件 (考点):

- 遗忘项 (Forgetting term): $-\eta y_j(n)^2 w_{ji}(n)$ 。这是防止权重无限增长的关键。

收敛性： 权重向量最终会收敛到单位长度 ($|w| = 1$)。

功能： 单个神经元使用 Oja 规则，能提取数据的第一主成分 (1st Principal Component) (方差最大的方向)。

10.1.3 紧缩法 (Deflation Method)

问题： Oja 规则只能找到第 1 主成分，如何找第 2、3...个？

算法逻辑： 从原始输入中减去已经找到的主成分信息。

公式：

$$\hat{x} = x - w_1 y = x - w_1 (w_1^T x)$$

步骤： 用“残差” $\hat{x}$ 作为输入，再次训练网络，得到的就是第 2 主成分。

10.2 第二板块：神经网络中的 PCA 与自动编码器

这部分是将数学原理转化为网络结构的实现方式。

10.2.1 自动编码器 (Auto-encoders)

定义： 一种前馈网络，目标是 $Output \approx Input$ 。

结构特征 (考点)：

- **瓶颈层 (Bottleneck Layer)：** 隐藏层节点数 $<$ 输入层节点数。
- **作用：** 强迫网络进行信息压缩，丢弃冗余特征，保留最显著的特征 (Efficient Code)。
- **Encoder (编码器)：** $Input \rightarrow Hidden$ 。
- **Decoder (解码器)：** $Hidden \rightarrow Output$ 。

与 PCA 的关系：

- 如果是线性激活函数，自动编码器的瓶颈层张成的子空间 = PCA 的前 m 个主成分子空间。
- 深度自动编码器 (多层) 可以学习非线性特征。

10.2.2 变体模型

Deep Auto-encoder (Hinton 06)： 多层结构，逐步压缩再逐步还原。

Denoising Auto-encoder (Vincent 08)：

- **机制：** 在输入加入随机噪声 ($x \rightarrow \tilde{x}$)。

- 目标：从损坏的输入 $\tilde{x}$ 重建原始输入 x 。
- 目的：学习更鲁棒 (Robust) 的特征，防止过拟合。

10.3 第三板块：聚类与 K-means

最经典的无监督算法，重点在于流程和逻辑困境。

10.3.1 K-means 算法流程 (背诵级)

1. 初始化：选定 K 个簇，随机初始化质心位置。
2. 分配 (Assignment): 计算每个点到质心的距离，将其分配给最近的质心。
3. 更新 (Update): 重新计算质心位置（取簇内所有点的平均值）。
4. 迭代：重复 2 和 3，直到收敛（质心不再移动）。

10.3.2 核心考点

目标函数：最小化簇内距离平方和 (Minimize intra-cluster variance)。

$$J = \sum \sum d^2(x_j, \mu_i)$$

局限性：

- 局部最优 (Local Optima): 结果严重依赖于初始质心的选择，不一定能找到全局最优解。
- 逻辑困境：未知质心位置就无法分配点，未分配点就无法算质心（这也是为什么要迭代的原因）。

10.4 第四板块：竞争学习 (Competitive Learning)

这是本 Lecture 的高潮，涉及生物学原理与解决“死神经元”问题。

10.4.1 赢家通吃 (Winner-Takes-All, WTA)

机制：输入模式进来后，所有输出神经元竞争，只有一个胜出（输出为 1），其余全输（输出为 0）。

生物学基础：侧抑制 (Lateral Inhibition) — 神经元抑制周围的邻居。

简单竞争学习规则：

$$\Delta w_{ji} = \eta y_j (x_i - w_{ji})$$

仅 (Only) 赢家更新权重！赢家向输入向量移动。

几何意义： 最终权重会停留在数据簇的质心位置。

10.4.2 “死单元”问题 (Dead Units) 与解决方案

现象： 某些神经元因为初始位置太差（太远），永远竞争不过别人，永远不更新。

解决方案 1： 泄漏学习 (Leaky Learning)

- 策略： 赢家大更新，输家小更新。
- 公式：
 - Winner: $w_{new} = w + \eta_w(x - w)$
 - Loser: $w_{new} = w + \eta_L(x - w)$ (其中 $\eta_w \gg \eta_L$)
- 效果： 缓慢地把“离群点”拖回数据密集区，让其有机会参赛。

10.4.3 Maxnet (WTA 的网络实现)

结构：

- 自连接 (Self-connection): 权重为 1（兴奋）。
- 互连接 (Mutual connection): 权重为 $-\epsilon$ （抑制）。

功能： 通过反复迭代，数值大的抑制数值小的，直到只剩一个正值。

10.4.4 墨西哥帽网络 (Mexican Hat Network)

超越 WTA： 不仅仅是单点胜出，而是区域互动。

连接逻辑（随距离变化）：

- 近邻 (Close): 合作/兴奋 ($w > 0$) $\rightarrow$ 形成活动泡。
- 远邻 (Distant): 竞争/抑制 ($w < 0$)。
- 极远 (Far): 无关 ($w = 0$)。

意义： 这是 SOM（自组织映射）的前身，引入了拓扑邻域的概念。

10.5 第五板块：应用 — 矢量量化 (Vector Quantization)

竞争学习的实际落地。

10.5.1 核心概念

码本 (Codebook): 也就是聚类中心/原型向量的集合 $\{m_1, \dots, m_K\}$ 。

压缩原理： 传输数据时，不传输整个向量，只传输它所属的类别索引 (Index)。

沃罗诺伊图 (Voronoi Diagram): 将空间划分为多个区域，每个区域内的点都归属于同一个质心。

10.6 考前叮嘱 (Exam Tips)

- **公式对比:** 考试时一定要看清题目问的是 Hebbian 还是 Oja。如果问 Hebbian 的缺点，必须回答 “Unbounded growth / Instability”。
- **死单元 (Dead Unit):** 这是一个高频名词解释或简答题。要能解释它产生的原因（初始位置差）以及解决办法（Leaky Learning）。
- **WTA vs. Mexican Hat:** WTA 是“你死我活”（单点），Mexican Hat 是“近邻互助，远邻对抗”（区域）。
- **Auto-encoder 原理:** 如果让你画图或解释，记得强调 Bottleneck 的作用是 “Information Compression” 或 “Feature Extraction”。

11 Lecture 11: 自组织映射与学习矢量量化 (SOM & LVQ)

11.1 第一板块：核心概念与生物学原理 (The Concept)

这一部分通常考察选择题或简答题，要求理解算法的设计动机。

11.1.1 核心目标 (Goal)

降维 (Dimensionality Reduction): 将任意高维的输入信号映射到 1 维或 2 维的离散输出网络上。

拓扑保持 (Topology Preserving): 这是 SOM 的灵魂。输入空间中相似（距离近）的样本，在输出地图上也必须位于相邻的位置。

口诀: Close in Input $\Rightarrow$ Close in Output.

11.1.2 生物学机制 (Mexican Hat Function)

大脑皮层的神经元连接遵循“墨西哥帽” (Mexican-hat function) 机制：

- **短程兴奋 (Short-range excitation):** 激活邻近的神经元（拉帮结派）。
- **长程抑制 (Long-range inhibition):** 抑制远处的神经元（排斥异己）。

SOM 通过“邻域函数”模拟了这个机制，从而形成簇 (Clusters)。

11.2 第二板块：SOM 算法流程与计算 (The Algorithm)

这是计算题和算法逻辑题的重灾区，所有公式必须背熟！

11.2.1 网络架构 (Architecture)

两层结构: 输入层 (Input) + 输出层 (Output/Lattice)。

全连接: 每个输入节点连接到所有输出神经元。

输出层结构: 通常是 2D 网格（矩阵），神经元在网格上的物理坐标决定了它们之间的距离（用于定义邻域）。

11.2.2 训练三大步 (The 3 Processes)

Step 1: 竞争 (Competition) — 找赢家

给定输入向量 x ，找到与它最匹配的神经元 j^* (Winner)。

判别公式 (**必须掌握**): 通常使用最小欧几里得距离:

$$j^* = \arg \min_j (\|x - w_j\|)$$

(备选方案: 最大化内积 $w_j^T x$, 前提是向量已归一化)

Step 2: 合作 (Cooperation) — 定邻域

不仅赢家更新, 赢家的邻居 (Neighbors) 也要更新。

邻域函数 $h_{j,i}(t)$:

- 通常是高斯函数 (Gaussian)。
- **最大值**: 在赢家中心。
- **衰减**: 随距离增加而衰减。

关键动态特性 (**必考点**): 邻域范围 $\sigma(t)$ 必须随时间收缩 (Shrink)。

- 初期 (Ordering Phase): 大邻域 $\rightarrow$ 建立全局结构。
- 后期 (Convergence Phase): 小邻域 $\rightarrow$ 精修局部细节。

Step 3: 适应 (Adaptation) — 改权重

SOM 权重更新通式 (**背下来!**):

$$w_j(t+1) = w_j(t) + \eta(t) \cdot h_{j,i}(t) \cdot (x - w_j(t))$$

参数解析:

- $w_j(t)$: 旧权重。
- $\eta(t)$: 学习率 (随时间衰减)。
- $h_{j,i}(t)$: 邻域函数 (决定更新力度, 赢家最大, 边缘最小)。
- $(x - w_j(t))$: 误差向量 (方向指向输入 x)。

物理意义: 将获胜者及其邻居的权重向量, 向输入向量 x 的方向拉动。

11.3 第三板块: SOM 的特性与结果 (Properties)

用于判断题或分析题, 描述训练后的网络状态。

11.3.1 近似输入空间 (Approximation)

SOM 的权重向量分布会近似原始数据的分布。

11.3.2 密度匹配 (Density Matching)

- 输入数据密集的区域 → 分配的神经元也多。
- 输入数据稀疏的区域 → 分配的神经元也少。

11.3.3 特征选择 (Feature Selection)

能够提取出数据潜在的统计特征。

11.4 第四板块：矢量量化与 LVQ (VQ & LVQ)

这是从无监督转为有监督的关键，容易混淆公式，请注意区分！

11.4.1 矢量量化 (Vector Quantization, VQ)

Voronoi 图：将空间划分为多个区域，每个区域由一个中心点 (Code vector) 代表。

规则：任何点归属于离它最近的中心点（谁近归谁）。

SOM 可以被视为一种带拓扑约束的无监督 VQ。

11.4.2 学习矢量量化 (LVQ) — 有监督扩展

区别：SOM 是无监督的，LVQ 是有监督 (Supervised) 的（利用类别标签 Class Labels）。

LVQ1 核心算法 (考试计算核心)：

输入 x （带标签），找到最近的权重 w （带标签）。

情况 A: 猜对了 (Labels Agree)

把 w 拉近 x （奖励）。

$$w^{new} = w^{old} + \eta(x - w)$$

情况 B: 猜错了 (Labels Disagree)

把 w 推远离 x （惩罚）。

$$w^{new} = w^{old} - \eta(x - w)$$

注意：这里没有邻域函数 h ，只有学习率 η 。只更新赢家 (Winner)。

11.4.3 LVQ2 (进阶)

同时更新两个原型向量（最近的正确类 + 最近的错误类），一个拉近，一个推远，主要用于优化边界。

11.4.4 LVQ 优缺点

优点: 简单、快、直观。

缺点: 对初始化敏感、对重叠类别处理不稳定。

11.5 考前特别提醒 (Final Check)

公式混淆警告:

- **SOM 更新:** 只有加号 (+)。目的是把大家都拉向输入数据（形成地形）。涉及邻域 h 。
- **LVQ 更新:** 有加有减 (+/-)。猜对是加（拉近），猜错是减（推远）。通常不涉及邻域 h 。

邻域策略: 记得解释为什么邻域要“先大后小”（先全局排序，后局部收敛）。

距离计算: 考试中如果需要手动计算找 Winner，通常用欧氏距离平方 ($\Delta x^2 + \Delta y^2$) 比较大小即可，不用非得开根号。

12 Lecture 12: 联想记忆与离散 Hopfield 网络

这是一份考试特化 (Exam-Specialized) 的知识点清单，涵盖了所有必须死记硬背的公式、必须理解的机制以及解题时的标准套路。

12.1 第一板块：联想记忆基础 (Associative Memory Basics)

这部分是概念题和选择题的高频区，重点在于区分不同类型的记忆机制。

12.1.1 核心定义与对比

内容寻址 (Content-Addressable): 联想记忆不依赖物理地址 (RAM style)，而是通过内容 (Content) 直接映射。

输入输出映射: 将输入模式 (Input Pattern) 映射到输出模式 (Output Pattern)。

12.1.2 两大分类 (必须要分清)

Auto-associative (自联想):

- 公式: $Input(x) \approx Output(y)$ (或理想情况下 $x = y$) 。
- 功能: 模式补全 (Completion)、去噪 (Recovery/Restoration)。
- 场景: 输入一张破损的人脸，输出一张完整的人脸。

Hetero-associative (异联想):

- 公式: $Input(x) \neq Output(y)$ 。
- 功能: 建立不同事物间的关联。
- 场景: 输入人脸 $\rightarrow$ 输出名字; 输入 $x \in R^n \rightarrow$ 输出 $y \in R^m$ (维度可以不同) 。

12.2 第二板块：简单联想记忆与矩阵数学 (Simple AM & Math)

这部分涉及基础的矩阵运算原理，是理解 Hopfield 权重构建的前置知识。

12.2.1 赫布学习规则 (Hebbian Learning Rule)

核心思想: 如果两个神经元同时激活 (ON) 或状态相同，它们之间的连接权重增强。

单样本权重公式: $\Delta w_{ij} = s_i \cdot t_j$

- 对于 Bipolar (-1/1): 符号相同则增，不同则减。
- 对于 Binary (0/1): 只有同时为 1 才增。

12.2.2 权重矩阵构建 (外积法 — Outer Product)

这是计算题的核心基础，必须熟练掌握矩阵乘法。

公式：

$$W = \sum_{p=1}^P s^T(p) \otimes t(p)$$

- s^T 是列向量， t 是行向量。
- 结果是一个 $n \times m$ 的矩阵。

口诀：列乘行，得矩阵。

12.2.3 回忆过程与干扰 (Recall & Cross-talk)

回忆公式： $Input \times W = Output$

$$s(k)W = s(k) \sum s^T(p)t(p)$$

数学展开 (考点)：

$$\text{Result} = \underbrace{s(k)s^T(k)t(k)}_{\text{Signal (主项)}} + \underbrace{\sum_{p \neq k} s(k)s^T(p)t(p)}_{\text{Noise (串扰项)}}$$

正交性 (Orthogonality)：

- 如果所有输入向量互相正交 ($s(k) \cdot s(p) = 0$)，则串扰项 (Cross-talk) 为 0，实现完美回忆。
- **考点：**随着存储模式数量 P 增加，正交性被破坏，噪声增加。

12.3 第三板块：离散 Hopfield 网络 (Discrete Hopfield Network)

这是本章最核心的考点，通常出现在大题中，要求你构建网络并进行迭代计算。

12.3.1 网络架构特征 (背诵)

结构：单层 (Single layer)，全连接 (Fully connected)。

节点：既是输入也是输出，只有两种状态 (1/-1 或 1/0)。

两大铁律 (权重约束)：

- 对称性 (Symmetric): $w_{ij} = w_{ji}$ 。
- 无自环 (Zero Diagonal): $w_{ii} = 0$ (对角线必须为0，否则网络失去纠错能力，输出恒等于输入)。

12.3.2 权重矩阵计算公式 (必考计算)

给定一组训练模式 $x^1, x^2, \dots, x^P$ ，构建权重矩阵 W ：

通用公式：

$$W = \left(\sum_{k=1}^P x^k (x^k)^T \right) - P \cdot I$$

操作步骤：

1. 将每个模式向量转为列向量 x 。
2. 计算每个模式的外积矩阵 $M_k = x \cdot x^T$ 。
3. 将所有 M_k 相加。
4. **关键动作**：把对角线全部写成 0（手动将对角线元素改为 0，或者减去 $P \times I$ ）。

检查点：矩阵必须是对称的。

12.3.3 状态更新规则 (Update Rule / Dynamics)

给定网络当前状态 $v(t)$ ，如何计算下一时刻状态？

净输入计算 (Net Input)：

$$H_i = \sum_{j \neq i} w_{ij} v_j(t) + I_i$$

（通常考试中 $I_i = 0$ ）

激活函数 (Sign Function)：

$$v_i(t+1) = \begin{cases} 1 & \text{if } H_i \geq 0 \\ -1 & \text{if } H_i < 0 \end{cases}$$

注意：有时题目会规定 $H_i = 0$ 时保持原状态不变，请仔细审题，若无特殊说明，通常 ≥ 0 取 1。

异步更新 (Asynchronous)：每次只随机选择一个节点进行更新。

重要考点：不同的更新顺序 (Update Sequence) 可能导致收敛到不同的稳定状态。

12.4 第四板块：系统性质与局限性 (Properties & Limits)

这部分通常涉及简答题或分析题，解释网络行为。

12.4.1 稳定性与收敛 (Convergence)

能量函数 (Energy Function): Hopfield 网络被视为动力系统，存在能量函数 E 。

结论: 在权重对称且无自环的条件下，网络能量随迭代单调递减，最终必然收敛到局部极小值 (Local Minimum)，即稳定状态 (Stable State) 或吸引子 (Attractor)。

基础记忆 (Fundamental Memories): 成功存储的模式就是吸引子。

12.4.2 伪状态 (Spurious States)

定义: 网络收敛到了一个未曾存储过的混合模式。

成因:

- 存储模式过多。
- 模式之间相关性太高（不满足正交性）。
- 记忆模式的线性组合或反转模式（如存储了 A ， $-A$ 也会成为稳定态）。

12.4.3 存储容量 (Capacity)

Hopfield 网络的存储能力是严重受限的。

经验公式: 最大存储量约为神经元数量的 15% ($P_{max} \approx 0.15N$)。

超出此限制将导致回忆完全失败。

12.5 第五板块：考试解题 SOP (Standard Operating Procedure)

如果考试出现 Hopfield 计算大题，请按此步骤书写，保证拿满步骤分：

12.5.1 Step 1: 检查模式类型

确认是 Binary (0/1) 还是 Bipolar (-1/1)。通常 Hopfield 使用 Bipolar。

12.5.2 Step 2: 构建 W 矩阵

1. 列出公式: $W = \sum xx^T - pI$ 。
2. 计算每个样本的外积。
3. 求和。
4. **关键动作:** 把对角线全部写成 0。
5. **检查点:** 矩阵必须是对称的。

12.5.3 Step 3: 进行迭代/回忆

给定输入向量 (Probe vector)。

如果是同步更新（罕见）：

- $V_{new} = \text{sgn}(W \cdot V_{old})$ ，矩阵向量乘法一次出结果。

如果是异步更新（常见）：

1. 指明更新的是哪个节点（例如 Node 4）。
2. 写出算式： $Net_4 = \sum w_{4j}v_j$ 。
3. 判断符号： $> 0 \rightarrow 1$ ， $< 0 \rightarrow -1$ 。
4. 更新状态向量，进行下一轮。

12.5.4 Step 4: 验证稳定性

当所有节点的输出状态在经过一轮更新后都不再发生变化（ $v(t+1) = v(t)$ ），则判定网络已收敛 (Converged)。

最终输出即为回忆结果。